Retrieval-Augmented Generation: A Deep Dive

1. Motivation

Large language models encode knowledge in their parameters during pre-training. However, this knowledge is static: it cannot be updated without costly retraining, and it is bounded by the training cutoff date. Retrieval-Augmented Generation (RAG) addresses these limitations by augmenting the model's context window with relevant documents retrieved at inference time. The result is a system that combines the fluency and reasoning of a generative model with the accuracy and currency of a search engine.

2. RAG Architecture

A standard RAG pipeline consists of three main components: an ingestion pipeline, a retrieval module, and a generation module. During ingestion, source documents are split into chunks, embedded using a dense encoder, and stored in a vector database. At query time, the user's question is embedded with the same encoder, and the nearest neighbors in the vector space are retrieved. These retrieved chunks are concatenated with the question into a prompt, which is passed to the generative LLM to produce the final answer.

3. Chunking Strategies

The choice of chunking strategy significantly impacts retrieval quality. Fixed-size chunking splits text every N characters with an optional overlap. Sentence-level chunking preserves semantic boundaries but may produce uneven chunk sizes. Recursive chunking attempts to split at paragraph boundaries first, falling back to sentence and word boundaries. Semantic chunking uses embedding similarity to group thematically related sentences. For most production systems, recursive splitting with 512-character chunks and 64-character overlap is a solid baseline.

4. Embedding Models

The embedding model determines how well semantic similarity is captured. Bi-encoder models (such as Sentence-BERT, E5, and GTE) encode queries and documents independently and compare them via dot product or cosine similarity, enabling efficient approximate nearest-neighbor search. Google's text-embedding-004 and gemini-embedding-exp-03-07 models offer state-of-the-art retrieval performance with 768 and 3072 dimensional output vectors respectively. The choice of model should balance quality, latency, and cost for the target use case.

5. Hybrid Retrieval

Pure dense retrieval excels at semantic matching but can miss exact keyword matches. Sparse retrieval methods such as BM25 excel at keyword matching but lack semantic understanding. Hybrid retrieval combines both signals using fusion algorithms such as Reciprocal Rank Fusion (RRF). RRF assigns a score of 1/(k + rank) to each result from each retriever and sums the scores across retrievers. With a typical k=60, RRF produces robust rankings that outperform either retriever in isolation on most benchmarks.

6. Reranking

After initial retrieval, a reranker model scores each retrieved chunk against the query using a cross-encoder architecture, which jointly encodes the query and document for higher accuracy than bi-encoders. Cohere's Rerank API, Jina Reranker, and BGE-Reranker are popular options. The reranker operates on a small candidate set (typically 20-50 chunks) and selects the top-k most relevant ones, improving precision without sacrificing recall.

7. Evaluation Metrics (RAGAS)

Metric	What It Measures	Ideal Score
Faithfulness	Answer grounded in context	1.0
Answer Relevancy	Answer addresses the question	1.0
Context Precision	Relevant chunks ranked first	1.0
Context Recall	Context covers ground truth	1.0